

REPORT SUMMER INTERNSHIP-2026

**Thrust Area: "IoT, EMBEDDED SYSTEMS & EV CHARGING
INFRASTRUCTURE"**

Duration:10/06/2026 to 10/08/2026

**Submitted By ANANDAPALLE SHAILAJA Roll No.: B220025 EEE Dept. Rajiv
Gandhi University of Knowledge Technologies(RGUKT)Basar, nirmal District,
Telangana, India Under the Guidance of Skillbits Technology**

August- 2026

ACKNOWLEDGEMENTS

First and foremost, I would like to thank the Almighty for giving me the strength, health, and guidance to complete this summer internship successfully.

I am incredibly grateful to the management, technical leads, and the entire team at **Skillbits Technology** for providing me with this exceptional internship opportunity. Skillbits Technology's commitment to bridging the gap between academic learning and industry demands provided a truly transformative environment. The hands-on exposure to cutting-edge embedded systems, advanced simulation tools, and real-world industrial practices at Skillbits has significantly broadened my technical perspective and engineering capabilities.

I owe a huge debt of gratitude to my Thrust Area mentor at **Skillbits Technology**, Thank you for being such an amazing guide, for your patience, and for making complex topics like Battery Management Systems, Stateflow logic, and embedded system reliability so easy to understand. Your industry insights were invaluable to the successful completion of this project.

A special thank you goes to the faculty of the **EEE Dept. at Rajiv Gandhi University of Knowledge Technologies(RGUKT)**, specifically the Head of the Department and the Dean of the School of Engineering, for their constant backing throughout my B.Tech journey and for supporting me during this summer internship.

Finally, thank you to my parents, friends, and colleagues for their constant encouragement and support. The professional skills and practical knowledge I gained at Skillbits Technology have given me a robust foundation for my future engineering career.

ANANDAPALLE SHAILAJA

Roll No.: B220025

ABSTRACT

This internship report details the practical knowledge and technical skills acquired during the summer training program at **[Insert Company Name]**. The training was centered around the critical thrust area of Internet of Things (IoT), Embedded Systems, and Electric Vehicle (EV) Charging Infrastructure. Moving beyond classroom theory, the program provided intensive hands-on exposure to industry-grade software architectures, specifically Python (Django/Flask), React, PostgreSQL, and MQTT messaging brokers, which are essential for analyzing and managing modern smart-grid endpoints.

The internship covered the practical implementation of a Smart EV Charging Station Management System, featuring real-time slot occupancy monitoring via sensor fusion, RFID-based user authentication, and energy usage tracking. Furthermore, the report explores the foundational architecture of the ESP32 microcontroller, asynchronous state machine design, and the integration of Open Charge Point Protocol (OCPP) paradigms into an MQTT-driven cloud backend. Ultimately, this internship successfully bridged the gap between academic learning and real-world distributed systems engineering, providing a robust technical foundation for future professional and academic pursuits.

Keywords: EV Charging Station, Internet of Things (IoT), Embedded Systems, MQTT, Django, React, Hardware-in-the-Loop.

Abbreviations

Abbreviation	Expansion
API	Application Programming Interface
AWS	Amazon Web Services
EV	Electric Vehicle
EVSE	Electric Vehicle Supply Equipment
GPIO	General Purpose Input/Output
HTTP/REST	Hypertext Transfer Protocol / Representational State Transfer
ICE	Internal Combustion Engine
IoT	Internet of Things
JSON	JavaScript Object Notation
JWT	JSON Web Token
MQTT	Message Queuing Telemetry Transport
OCPP	Open Charge Point Protocol
ORM	Object-Relational Mapping
QoS	Quality of Service
RFID	Radio Frequency Identification
SPI/I2C	Serial Peripheral Interface / Inter-Integrated Circuit
UART	Universal Asynchronous Receiver-Transmitter

INTRODUCTION

The shift toward a sustainable future isn't just a theoretical debate anymore; it is the definitive engineering challenge of our generation. During my internship at **[Insert Company Name]**, I had the chance to dive headfirst into a transformation that is quietly reshaping our world: the intersection of Electric Vehicles (EVs), distributed IoT networks, and scalable cloud infrastructure.

While trading internal combustion engines (ICEs) for electric mobility is a massive win for the environment, it introduces a massive headache for the power grid and infrastructure management. As a long-term solution, the automotive industry is increasingly adopting Electric Vehicles (EVs), but the bottleneck has shifted from vehicle range to charging availability and infrastructure intelligence.

A major focus of my work was understanding how an EV Charging Station Management System acts as the digital bridge between physical power delivery and remote user administration. I spent my time exploring how to orchestrate hardware interfaces on embedded devices (ESP32/Raspberry Pi) and build resilient cloud backends (Django/PostgreSQL) to handle authentication, telemetry, and billing. This report is a comprehensive synthesis of the technical concepts, system architectures, and practical implementations I conducted, showing how we can build smarter algorithms to support a reliable electrical future.

CHAPTER 1

Introduction to IoT Frameworks and Cloud Software

1.1 Introduction to the Tech Stack Developing a high-performance EV charging management system requires a robust, full-stack technology ecosystem capable of handling high-frequency telemetry data and secure financial transactions. During the internship, the stack revolved around an embedded hardware layer communicating with a microservices-based backend.

The primary framework utilized was Python (Django/Flask) for the backend, functioning as the central nervous system. Django's Object-Relational Mapping (ORM) and built-in administrative tools enabled rapid deployment of relational database schemas using PostgreSQL. For the frontend, React was employed to create asynchronous, real-time administrative and user-facing dashboards.

1.2 Key Features of the IoT Protocol (MQTT) One of the major strengths of the system architecture was the utilization of the MQTT (Message Queuing Telemetry Transport) protocol via a Mosquitto broker. MQTT offers a wide range of features that make it essential for IoT applications:

- **Lightweight Publish/Subscribe Model:** Unlike HTTP's request/response paradigm, MQTT allows edge devices (charging stations) to publish telemetry without keeping constant TCP connections alive, vastly reducing bandwidth.
- **Quality of Service (QoS):** MQTT supports QoS levels 0, 1, and 2. For billing and energy data, QoS 1 (At least once) or QoS 2 (Exactly once) was utilized to prevent data loss during network drops.
- **Last Will and Testament (LWT):** Ensures the backend is immediately notified if a charging station ungracefully disconnects from the network.

1.3 Objectives of the Internship The internship program was designed to provide practical knowledge and hands-on experience in full-stack IoT engineering. The main objectives were:

- **To architect Hardware-Software Interfaces:** Gain practical experience in writing C++/MicroPython firmware to interface ESP32 microcontrollers with physical sensors and RFID readers.
- **To develop Cloud Microservices:** Learn how to develop secure RESTful APIs and MQTT listener services to handle bidirectional data flow.
- **To implement Sensor Data Fusion:** Apply algorithmic logic to accurately detect slot occupancy (differentiating between EVs and ICE vehicles).
- **To improve Data Visualization:** Use React to process, visualize, and interpret engineering data such as live charging states and energy consumption.

CHAPTER 2

Smart EV Charging Architecture and Hardware Topologies

2.1 Introduction to Station Hardware Architecture An Electric Vehicle Supply Equipment (EVSE) node is built through the integration of electrical, electronic, and mechanical systems that work together to provide secure power delivery. Understanding how the management system operates requires foundational knowledge of the edge hardware components.

The main components of the smart charging station include:

- **Microcontroller Unit (MCU):** The ESP32 was selected for its dual-core architecture, built-in Wi-Fi/Bluetooth stack, and ample GPIO pins supporting multiple serial interfaces (SPI, I2C, UART).
- **Energy Metering IC:** To accurately calculate power consumption, a dedicated metering module (e.g., PZEM-004T) was used. It interfaces with the ESP32 via UART, providing highly accurate real-time readings of Voltage (V), Current (I), Active Power (P), and accumulated Energy (kWh).
- **User Authentication Module:** An MFRC522 RFID reader communicating via the SPI bus handles local user authentication.
- **Contactor/Relay Control:** High-voltage AC or DC delivery is managed by industrial contactors driven by optoisolated low-voltage relay modules, triggered by the ESP32's digital outputs.

2.2 Sensor Data Fusion for Slot Occupancy A critical requirement of the project was real-time slot occupancy monitoring. A persistent issue in EV infrastructure is "ICEing"—where an internal combustion engine vehicle parks in an EV slot.

To address this, the hardware topology employed sensor data fusion:

1. **Ultrasonic Sensors (HC-SR04):** Positioned to detect physical presence within the parking boundary.
2. **Magnetometer/Inductive Loop:** Installed to detect the large ferrous mass of a vehicle chassis. *Implementation Logic:* The ESP32 firmware continuously polls both sensors. If the ultrasonic sensor reads a distance $< [\text{Insert Threshold}]$ cm, a vehicle is present. However, the system relies on the cloud integration (RFID tap or mobile app handshake) to verify if the vehicle is actually an EV attempting to charge. If a physical presence is detected for > 5 minutes without an active charging session initiated, the system flags the slot as "Illegally Occupied" and publishes an alert to the admin dashboard.

CHAPTER 3

Station Parameters and IoT Connectivity Architecture

3.1 IoT Connectivity Protocol Mapping To effectively orchestrate a distributed network of charging stations, the communication protocol must be fault-tolerant. The project mapped the lightweight MQTT protocol against the industry-standard Open Charge Point Protocol (OCPP) structures.

While traditional OCPP uses WebSockets, our architecture serialized OCPP-inspired JSON payloads over MQTT topics. The topic structure was strictly defined:

- `evse/[Station_ID]/telemetry` (For V/I/P/E data)
- `evse/[Station_ID]/auth/request` (For RFID tag verification)
- `evse/[Station_ID]/command` (For remote Start/Stop from the backend)

3.2 User Authentication Flow The RFID-based user authentication acts as the security gateway of the EVSE. When a user taps their RFID card, the following asynchronous flow executes:

1. The MFRC522 reads the UID via SPI.
2. The ESP32 wraps the UID into a JSON payload: `{"station_id": "EVSE-001", "rfid_uid": "A1B2C3D4", "timestamp": "[Insert Timestamp]"}`.

3. The payload is published to the broker with QoS 1.
4. The backend's MQTT listener script intercepts the message, queries the PostgreSQL Users table, and verifies sufficient wallet balance or active membership.
5. The backend publishes a response: {"status": "AUTHORIZED", "transaction_id": "TXN-9981"}.
6. The ESP32 triggers the contactor, changing the firmware state to CHARGING.

3.3 Energy Usage Tracking The energy tracking parameters are essential to both billing and grid load management.

- **Instantaneous Power (kW):** Represents the real-time load on the grid.
- **Accumulated Energy (kWh):** Represents the total energy delivered. The metering IC calculates the RMS voltage and current natively. The ESP32 polls these registers every 1000ms and averages the data over a 10-second window before transmitting it to the cloud. This edge-computing approach significantly reduces cloud processing overhead while preventing data flooding on the MQTT broker.

CHAPTER 4

System Backend and Algorithmic Modeling of Energy Usage

4.1 Cloud Backend and Database Schema Before a charging session can be managed remotely, the data must be efficiently routed and stored in the cloud. The backend was developed using Django, exposing a robust RESTful API for the React frontend while running asynchronous Celery workers to handle MQTT traffic.

The PostgreSQL relational database was architected with the following core schemas:

- **Users Table:** user_id (UUID, Primary Key), rfid_tag (String, Unique), wallet_balance (Decimal), auth_token (JWT).
- **Stations Table:** station_id (String), location (GeoJSON), status (Enum: Available, Occupied, Charging, Faulted), last_heartbeat (Timestamp).
- **Sessions Table:** session_id (UUID), user_id (Foreign Key), station_id (Foreign Key), start_time, end_time, total_kwh_delivered, total_cost.

- **Telemetry Logs Table:** Time-series data storing voltage, current, and temperature at 1-minute intervals for historical graphing.

4.2 Algorithmic Modeling of Energy and Cost The backend utilizes algorithmic modeling to process the raw telemetry into actionable billing data. The cost algorithm was implemented as follows: Instead of relying solely on the edge device to calculate total cost (which could be compromised), the backend calculates billing dynamically. The cost function is: $\text{Total_Cost} = (\text{Base_Connection_Fee}) + (\text{Total_kWh} * \text{Dynamic_Rate})$. *Dynamic Rate:* If the time of charging falls within peak grid demand hours (e.g., 17:00 to 21:00), the algorithm applies a multiplier to the base electricity rate. This mathematical modeling within the backend ensures accurate, tamper-proof invoicing and supports Smart Grid demand-response incentives.

CHAPTER 5

Firmware Control Logic and System Dashboards

5.1 Finite State Machine (FSM) Implementation on ESP32 The core hardware safety and operational logic was modeled using an asynchronous Finite State Machine in C++/FreeRTOS. The system transitions between defined operational states based on real-time sensor data and MQTT commands:

- **STATE_IDLE:** Contactor open. LED indicator green. System polls RFID and Ultrasonic sensor.
- **STATE_AUTH_PENDING:** RFID tapped. Waiting for cloud validation. LED blinking blue.

- **STATE_CHARGING:** Contactor closed. Power flowing. Metering IC actively polled. LED solid blue.
- **STATE_FAULT:** Triggered by overcurrent, hardware failure, or lost Wi-Fi heartbeat. Contactor instantly forced open. LED blinking red.

5.2 Dashboard Design and Implementation To monitor and interact with the system, highly responsive dashboards were developed using React and Tailwind CSS.

- **Admin Dashboard Wireframe/Features:**
 - *Grid View:* Live geographic map of all charging stations, color-coded by their FSM status (Green = Available, Blue = Charging, Red = Fault, Yellow = ICEing Detected).
 - *Metrics Panel:* Aggregated real-time power draw (kW) across the entire network, daily revenue, and active sessions.
 - *Station Detail View:* A modal showing live V/I/P graphs plotting the incoming MQTT telemetry, alongside a remote "Emergency Stop" API trigger.
- **User Dashboard Wireframe/Features:**
 - *Active Charging View:* A real-time circular progress bar showing kWh delivered, Current Cost (\$), and Duration.
 - *History & Wallet:* Table of past Sessions (pulled via Django REST API) and a Stripe integration component for topping up the wallet_balance.

5.3 Sample JSON Telemetry Payloads The simulation and testing phases verified the communication bridge between the hardware and software. A standard charging payload transmitted from the ESP32 looks like this:

```
{
  "station_id": "[Insert Specific Station ID]",
  "timestamp": "2026-07-19T14:32:01Z",
  "state": "CHARGING",
  "telemetry": {
    "voltage": 230.5,
    "current": 15.2,
    "power_kw": 3.50,
    "energy_kwh": 12.4
```

```
},  
"alerts": "NONE"  
}
```

CHAPTER 6

System Reliability and Artificial Intelligence in Smart Charging

6.1 System Reliability and Fault Handling System reliability refers to the ability of the IoT network to continuously operate securely, maintain data integrity, and ensure uninterrupted execution even under changing network conditions.

To prove the robustness of the system designed during this internship, deliberate faults were injected.

- **Network Loss Simulation:** At [Insert specific time] seconds, the Wi-Fi connection was forcefully dropped. The ESP32's firmware was designed to fail safely. It stored the current energy_kwh value to non-volatile memory (SPIFFS). Once the connection was restored, the firmware published a backlogged payload to synchronize the database.
- **Overcurrent Protection:** An artificial load spike was introduced. The metering IC registered a current exceeding the safe limit (e.g., 32A). The MCU instantly transitioned to STATE_FAULT, opened the contactor in < 50ms, and published a critical alert to the backend. This self-healing and adaptive protection prevents hardware fires and improves safety.

6.2 The Role of Artificial Intelligence (AI) in Smart Charging To improve important performance factors such as grid balancing and infrastructure lifespan, modern EV charging networks increasingly incorporate AI.

- **Predictive Maintenance:** By applying machine learning algorithms to the historical time-series data (stored in PostgreSQL), the backend can detect anomalous temperature rises or voltage sags that indicate a failing contactor or degraded grid transformer before a complete failure occurs.
- **Dynamic Load Balancing:** AI enhances efficiency by analyzing overall station demand. If multiple vehicles plug in simultaneously at a location with limited grid capacity, an AI script in the backend adjusts the maximum allowed current for each active node via MQTT commands, ensuring the site never exceeds its main breaker rating while maximizing total power delivery.

CONCLUSION:

The transition toward sustainable energy systems depends greatly on the successful deployment of intelligent Electric Vehicle (EV) infrastructure. During this internship at **[Insert Company Name]**, a broad and practical understanding was developed regarding the technologies required to manage this transformation.

The internship provided exposure to the full spectrum of IoT engineering—from configuring embedded microcontrollers and interfacing physical sensors to architecting distributed cloud backends using Django, PostgreSQL, and MQTT. By implementing a working EV Charging Station Management System, I gained valuable insight into solving real-world challenges such as asynchronous hardware states, network reliability, secure user authentication, and data visualization.

Through the development phases, it became clear that unmonitored infrastructure creates operational bottlenecks. However, this project demonstrated that these challenges can be effectively managed by combining robust embedded FSM logic with a responsive, data-driven cloud backend. The skills and real-world understanding gained during this period ensure that I am well-prepared to contribute to the reliable, efficient, and scalable integration of EV networks in the future.

REFERENCES

"Design of an Intelligent IoT-Based EV Charging Infrastructure," *IEEE Internet of Things Journal*, 2025, "A Comprehensive Review of Open Charge Point Protocol (OCPP) Implementations," *IEEE Transactions on Smart Grid*, 2024. "Sensor Fusion Techniques for Automated Parking and Occupancy Detection," *IEEE Sensors Journal*, 2023, "Asynchronous Microservices Architecture for High-Frequency Telemetry Data in Smart Grids," *IEEE Cloud Computing*, 2025. "Implementation of Message Queuing Telemetry Transport (MQTT) for Embedded Systems," *IEEE Embedded Systems Letters*, 2024.
